reporting_milestone2

November 13, 2026

1 Executive Summary (AI GENERATED WITH OPENAI)

This milestone evaluates the performance, cost, and latency of four foundation models available through AWS Bedrock: Amazon Nova Lite, NVIDIA Nemotron Nano 12B, Qwen3 32B, and Llama 4 Scout 17B. The models were tested across four representative e-commerce workloads: customer issue classification, customer response drafting, product catalogue extraction, and product review summarization. Each model completed 25 evaluations per workload, resulting in 400 evaluations overall.

The results indicate that all four models achieved relatively high quality scores, although differences emerged across workloads. Customer response drafting and product review summarization produced consistently strong results, with average scores generally exceeding 4.8 out of 5. Greater variation was observed in customer issue classification and product catalogue extraction, where individual errors and differences in information completeness had a more noticeable effect on performance.

Cost analysis revealed substantial differences between models. Amazon Nova Lite was the least expensive option across all four workloads, with estimated costs ranging from approximately $53 to $151 per million requests. Other models were frequently more than twice as expensive despite relatively small improvements in average quality. Latency was generally comparable across models, although Nova Lite exhibited notably higher average latency for customer response drafting.

These findings suggest that selecting a more expensive model does not necessarily produce proportional improvements in quality, particularly for relatively simple tasks. However, the narrow distribution of high evaluation scores also raises questions about the difficulty of the test cases and the ability of the automated evaluation model to distinguish performance differences.

Overall, the results demonstrate the importance of workload-specific model selection and provide an initial empirical foundation for balancing quality, cost, and latency. The next milestone will focus on improving evaluation reliability, introducing more challenging test cases, and developing a model-selection strategy that accounts for operational constraints and business priorities.

[28]:
```
import pandas as pd
from pathlib import Path

ROOT = Path.cwd().parent

INPUT_FILE = ROOT / 'data' / 'pricing' / 'bedrock_pricing.csv'
```

[29]:
```
df_price = pd.read_csv(INPUT_FILE)
```

Here we have the prices per model for 1million tokens on output and input. These will be used to quantify the costs of using these models and then extrapolated to the fictional workloads they will be applied to, which helps us better understand the cost at scale. We purposely chose cheap models and simple tasks to keep our personal costs low for this project as we will be paying out of pocket for the bedrock fees.

[30]:
```
df_price
```

[30]:
```
                 model_name bedrock_model_pricing_name  \
0  nvidia_nemotron_nano_12b  NVIDIA Nemotron Nano 2 VL
1                 nova_lite           Amazon Nova Lite
2                 qwen3_32b                  Qwen3 32B
3           llama_scout_17b          Llama 4 Scout 17B

                              region  input_price_usd_per_1m_tokens  \
0  US East (N. Virginia) / us-east-1                            0.20
1  US East (N. Virginia) / us-east-1                            0.06
2  US East (N. Virginia) / us-east-1                            0.15
3  US East (N. Virginia) / us-east-1                            0.17

   output_price_usd_per_1m_tokens
0                            0.60
1                            0.24
2                            0.60
3                            0.66
```

First we looked at overall summary scores for the tasks on our key metrics.

[31]:
```
df_issue_classification = pd.read_csv(ROOT / 'data' / 'outputs' /␣
 ↪'customer_issue_classification_model_comparison.csv')
df_response_drafting = pd.read_csv(ROOT / 'data' / 'outputs' /␣
 ↪'customer_response_drafting_model_comparison.csv')
df_catalogue_extraction = pd.read_csv(ROOT / 'data' / 'outputs' /␣
 ↪'product_catalogue_extraction_model_comparison.csv')
df_review_summarization = pd.read_csv(ROOT / 'data' / 'outputs' /␣
 ↪'product_review_summarization_model_comparison.csv')
```

We merge in the pricing data to the task data

[32]:
```
df_tasks = pd.concat([df_issue_classification, df_response_drafting,␣
 ↪df_catalogue_extraction, df_review_summarization])
df_tasks = df_tasks.merge(df_price[['model_name',␣
 ↪'input_price_usd_per_1m_tokens', 'output_price_usd_per_1m_tokens']],␣
 ↪how='left', on='model_name')
```

[33]:
```
df_tasks.head()
```

[33]:
```
                        workload                 model_name  \
0  customer_issue_classification  nvidia_nemotron_nano_12b
1  customer_issue_classification                 nova_lite
2  customer_issue_classification                  qwen3_32b
3  customer_issue_classification           llama_scout_17b
4     customer_response_drafting                 nova_lite

   average_quality_score  successful_evaluations  average_latency_seconds  \
0                   4.82                      25                     0.95
1                   4.64                      25                     0.73
2                   4.61                      25                     0.60
3                   4.56                      25                     0.95
4                   4.99                      25                     4.04

   average_input_tokens  average_output_tokens  input_price_usd_per_1m_tokens  \
0                585.68                  99.36                           0.20
1                618.80                  66.68                           0.06
2                570.04                  69.84                           0.15
3                577.08                 102.24                           0.17
4               1569.48                 235.24                           0.06

   output_price_usd_per_1m_tokens
0                            0.60
1                            0.24
2                            0.60
3                            0.66
4                            0.24
```

We then use our pricing data to calculate cost figures for comparison - we use 1 million tasks as our estimate of a high volume workload that a company like amazon might undergo on a daily or weekly basis - this is just a random guess.

[39]:
```
def calculate_avg_cost(row):
    input_cost = (row['average_input_tokens'] / 1_000_000) *␣
 ↪row['input_price_usd_per_1m_tokens']
    output_cost = (row['average_output_tokens'] / 1_000_000) *␣
 ↪row['output_price_usd_per_1m_tokens']
    return input_cost + output_cost

def calculate_total_cost(row):
    input_cost = (row['average_input_tokens'] / 1_000_000) *␣
 ↪row['input_price_usd_per_1m_tokens'] * row['successful_evaluations']
    output_cost = (row['average_output_tokens'] / 1_000_000) *␣
 ↪row['output_price_usd_per_1m_tokens']  * row['successful_evaluations']
    return input_cost + output_cost

def calculate_cost_per(row, n=1000000):
```

```
    input_cost = (row['average_input_tokens'] / 1_000_000) *␣
 ↪row['input_price_usd_per_1m_tokens'] * n
    output_cost = (row['average_output_tokens'] / 1_000_000) *␣
 ↪row['output_price_usd_per_1m_tokens']  * n
    return input_cost + output_cost

df_tasks['avg_cost'] = df_tasks.apply(calculate_avg_cost, axis=1)
df_tasks['total_cost'] = df_tasks.apply(calculate_total_cost, axis=1)
df_tasks['cost_per_1mill'] = df_tasks.apply(calculate_cost_per, axis=1)
```

[40]:
```
df_tasks
```

[40]:

	workload	model_name	\
0	customer_issue_classification	nvidia_nemotron_nano_12b	
1	customer_issue_classification	nova_lite	
2	customer_issue_classification	qwen3_32b	
3	customer_issue_classification	llama_scout_17b	
4	customer_response_drafting	nova_lite	
5	customer_response_drafting	qwen3_32b	
6	customer_response_drafting	nvidia_nemotron_nano_12b	
7	customer_response_drafting	llama_scout_17b	
8	product_catalogue_extraction	llama_scout_17b	
9	product_catalogue_extraction	nvidia_nemotron_nano_12b	
10	product_catalogue_extraction	qwen3_32b	
11	product_catalogue_extraction	nova_lite	
12	product_review_summarization	llama_scout_17b	
13	product_review_summarization	qwen3_32b	
14	product_review_summarization	nova_lite	
15	product_review_summarization	nvidia_nemotron_nano_12b	

	average_quality_score	successful_evaluations	average_latency_seconds	\
0	4.82	25	0.95	
1	4.64	25	0.73	
2	4.61	25	0.60	
3	4.56	25	0.95	
4	4.99	25	4.04	
5	4.99	25	1.04	
6	4.97	25	1.38	
7	4.96	25	1.42	
8	4.51	25	1.43	
9	4.28	25	1.27	
10	4.26	25	1.03	
11	4.18	25	1.10	
12	4.89	25	1.05	
13	4.88	25	0.99	
14	4.85	25	1.39	
15	4.85	25	1.08	

	average_input_tokens	average_output_tokens	\
0	585.68	99.36	
1	618.80	66.68	
2	570.04	69.84	
3	577.08	102.24	
4	1569.48	235.24	
5	1332.72	199.00	
6	1412.40	198.56	
7	1324.32	193.64	
8	1278.28	310.56	
9	1358.08	215.40	
10	1283.72	209.32	
11	1517.76	225.80	
12	354.40	141.68	
13	338.48	164.68	
14	335.00	166.84	
15	349.32	152.72	

	input_price_usd_per_1m_tokens	output_price_usd_per_1m_tokens	avg_cost	\
0	0.20	0.60	0.000177	
1	0.06	0.24	0.000053	
2	0.15	0.60	0.000127	
3	0.17	0.66	0.000166	
4	0.06	0.24	0.000151	
5	0.15	0.60	0.000319	
6	0.20	0.60	0.000402	
7	0.17	0.66	0.000353	
8	0.17	0.66	0.000422	
9	0.20	0.60	0.000401	
10	0.15	0.60	0.000318	
11	0.06	0.24	0.000145	
12	0.17	0.66	0.000154	
13	0.15	0.60	0.000150	
14	0.06	0.24	0.000060	
15	0.20	0.60	0.000161	

	total_cost	cost_per_100k	cost_per_1mill
0	0.004419	17.67520	176.7520
1	0.001328	5.31312	53.1312
2	0.003185	12.74100	127.4100
3	0.004140	16.55820	165.5820
4	0.003766	15.06264	150.6264
5	0.007983	31.93080	319.3080
6	0.010040	40.16160	401.6160
7	0.008823	35.29368	352.9368
8	0.010557	42.22772	422.2772

```
9     0.010021       40.08560        400.8560
10    0.007954       31.81500        318.1500
11    0.003631       14.52576        145.2576
12    0.003844       15.37568        153.7568
13    0.003739       14.95800        149.5800
14    0.001504        6.01416         60.1416
15    0.004037       16.14960        161.4960
```

Here we install seaborn to help with visualizations

[ ]:
```
#!pip install matplotlib seaborn
```

```
Requirement already satisfied: matplotlib in /home/kenneth/Documents/courses/mia
_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(3.11.2)
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: contourpy>=1.0.1 in /home/kenneth/Documents/cours
es/mia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /home/kenneth/Documents/courses/m
ia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /home/kenneth/Documents/cour
ses/mia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-
packages (from matplotlib) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/kenneth/Documents/cour
ses/mia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-
packages (from matplotlib) (1.5.1)
Requirement already satisfied: numpy>=1.25 in /home/kenneth/Documents/courses/mi
a_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from matplotlib) (2.5.4)
Requirement already satisfied: packaging>=20.0 in /home/kenneth/Documents/course
s/mia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in /home/kenneth/Documents/courses/mia_
5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages (from
matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in /home/kenneth/Documents/courses/m
ia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from matplotlib) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in /home/kenneth/Documents/c
ourses/mia_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-
packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in /home/kenneth/Documents/courses/mi
a_5130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages
(from seaborn) (3.1.1)
Requirement already satisfied: six>=1.5 in /home/kenneth/Documents/courses/mia_5
```

```
130_grp7_model_performance_comparison/.venv/lib/python3.12/site-packages (from
python-dateutil>=2.7->matplotlib) (1.17.0)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

The visualizations tell an interesting story. Mainly that the spread in evaluation quality is actually quite narrow. This is likely because the tasks were too simple and the ability for even our cheap models to do the task was trivial. The other aspect worth exploring is that our judge model was also a cheap model - which could mean it just didn't do a good job as a judge and thought every model performed well. A follow up for next milestone is to use a medium tier model for the evaluation of the budget models we used.

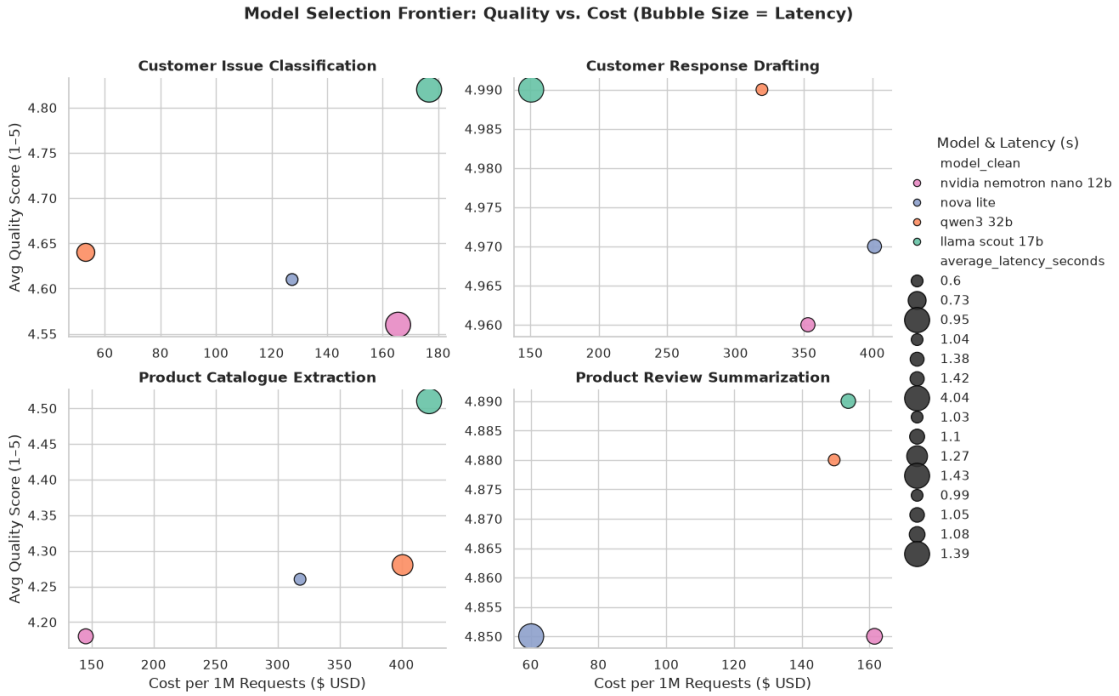
[49]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

df_tasks["workload_clean"] = df_tasks["workload"].str.replace("_", " ").str.
 ↪title()
df_tasks["model_clean"] = df_tasks["model_name"].str.replace("_", " ")

sns.set_theme(style="whitegrid", context="notebook")
palette = sns.color_palette("Set2", n_colors=df_tasks["model_clean"].nunique())

g = sns.FacetGrid(
    df_tasks,
    col="workload_clean",
    col_wrap=2,
    height=4.2,
    aspect=1.3,
    sharex=False,
    sharey=False,
)
g.map_dataframe(
    sns.scatterplot,
    x="cost_per_1mill",
    y="average_quality_score",
    hue="model_clean",
    size="average_latency_seconds",
    sizes=(100, 450),
    palette=palette,
    alpha=0.9,
    edgecolor="black",
)
g.set_titles("{col_name}", fontweight="bold")
g.set_axis_labels("Cost per 1M Requests ($ USD)", "Avg Quality Score (1-5)")
g.add_legend(title="Model & Latency (s)")
plt.subplots_adjust(top=0.88)
g.fig.suptitle(
```

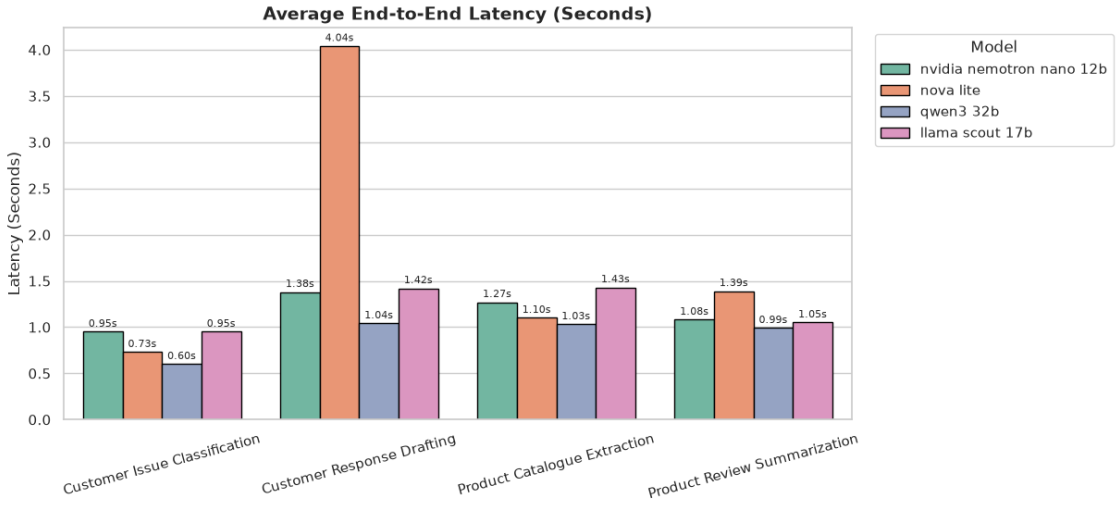
```
    "Model Selection Frontier: Quality vs. Cost (Bubble Size = Latency)",
    fontsize=14,
    fontweight="bold",
)
plt.savefig("1_cost_quality_frontier.png", dpi=300, bbox_inches="tight")
plt.show()
```



Latency was quite uniform in most cases, excluding for some reason Nova lite had a long response time for customer response drafting. Perhaps the model struggled with this task as it may have been more taxing - which again suggests perhaps it might have not been up to the task of being a judge.

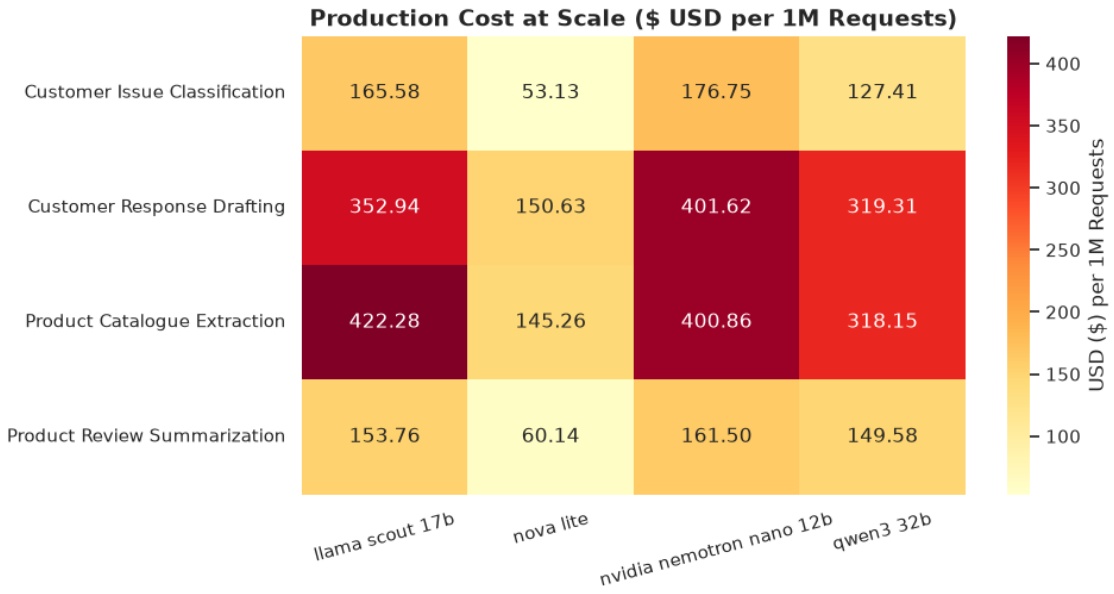
[51]:
```
plt.figure(figsize=(11, 5.5))
ax = sns.barplot(data=df_tasks, x="workload_clean",␣
 ↪y="average_latency_seconds", hue="model_clean", palette=palette,␣
 ↪edgecolor="black")
plt.title("Average End-to-End Latency (Seconds)", fontsize=14,␣
 ↪fontweight="bold")
plt.xlabel("")
plt.ylabel("Latency (Seconds)")
plt.xticks(rotation=15)
plt.legend(title="Model", bbox_to_anchor=(1.02, 1), loc="upper left")
for container in ax.containers:
    ax.bar_label(container, fmt="%.2fs", fontsize=8, padding=2)
plt.savefig("2_latency.png", dpi=300, bbox_inches="tight")
```

```
plt.show()
```



The cost at scale for the models showed that Nova lite was the cheapest by far on any tasks and the other models had over double the at scale costs. This begs the question - is the additional cost worth the potential lift in quality from these other models?

[52]:
```
pivot_cost = df_tasks.pivot(index="workload_clean", columns="model_clean",␣
↪values="cost_per_1mill")
plt.figure(figsize=(9, 5))
sns.heatmap(pivot_cost, annot=True, fmt=".2f", cmap="YlOrRd", cbar_kws={"label":
↪ "USD ($) per 1M Requests"})
plt.title("Production Cost at Scale ($ USD per 1M Requests)", fontsize=14,␣
↪fontweight="bold")
plt.xlabel("")
plt.ylabel("")
plt.xticks(rotation=15)
plt.savefig("3_cost_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
```



Next we look at the distributions of the evaluations to better understand the variation observed in the metrics and investigate outliers.

[58]:
```
df_full = pd.read_csv(ROOT / 'data' / 'outputs' / 'all_workloads_distributions.
↪csv')
```

[59]:
```
df_full.head()
```

[59]:

	workload	model_name	input_data	generated_output	overall_score
0	product_catalogue_extraction	nova_lite	{"brand": "[{'language_tag': 'en_CA', 'value':…	```json\n{\n "product_id": "B07SX8KYL3",\n …	4.6
1	product_catalogue_extraction	nvidia_nemotron_nano_12b	{"brand": "[{'language_tag': 'en_CA', 'value':…	{\n "product_id": "B07SX8KYL3",\n "produ…	4.4
2	product_catalogue_extraction	qwen3_32b	{"brand": "[{'language_tag': 'en_CA', 'value':…	{\n "product_id": "B07SX8KYL3",\n "produ…	4.4
3	product_catalogue_extraction	llama_scout_17b	{"brand": "[{'language_tag': 'en_CA', 'value':…	Here is the extracted product information in t…	4.6
4	product_catalogue_extraction	nova_lite	{"brand": "[{'language_tag': 'en_CA', 'value':…	```json\n{\n "product_id": "B07YF2L8YK",\n …	4.2

```
                                    overall_comment judge_model  \
0  The model performed excellently in extracting …   nova_lite
1  The model performed well in accurately extract…   nova_lite
2  The model performed excellently in terms of ac…   nova_lite
3  The model-generated output is highly accurate,…   nova_lite
4  The model performed well in extracting accurat…   nova_lite

   judge_latency_seconds  latency_seconds  input_tokens  …  Empathy_score  \
0                 1.9158           1.2314          1949  …            NaN
1                 1.7072           1.4529          1681  …            NaN
2                 1.5173           1.1592          1570  …            NaN
3                 1.8027           1.1985          1561  …            NaN
4                 1.5617           1.2085          1039  …            NaN

  Empathy_reason  Professionalism_score Professionalism_reason  \
0            NaN                    NaN                    NaN
1            NaN                    NaN                    NaN
2            NaN                    NaN                    NaN
3            NaN                    NaN                    NaN
4            NaN                    NaN                    NaN

   Helpfulness_score Helpfulness_reason  Policy Compliance_score  \
0                NaN                NaN                      NaN
1                NaN                NaN                      NaN
2                NaN                NaN                      NaN
3                NaN                NaN                      NaN
4                NaN                NaN                      NaN

  Policy Compliance_reason  Factuality_score Factuality_reason
0                      NaN               NaN               NaN
1                      NaN               NaN               NaN
2                      NaN               NaN               NaN
3                      NaN               NaN               NaN
4                      NaN               NaN               NaN

[5 rows x 47 columns]
```

Of interest is the judges reasoning for giving highest or lowest marks across the various tasks. There does seem to be legitimate reasoning for giving lower scores. A human review should verify whether these are warrented and if the judge was up to the task.

[66]:
```
cols = ["workload", "model_name", "overall_score", "overall_comment"]

# Ensure scores are numeric
df_full["overall_score"] = pd.to_numeric(
    df_full["overall_score"], errors="coerce"
```

```
)

# Sort by score, highest first
df_sorted = df_full.sort_values("overall_score", ascending=False)

# Top 5 evaluations per workload
top_5 = (
    df_sorted.groupby("workload", sort=False)
    .head(5)[cols]
    .reset_index(drop=True)
)

# Bottom 5 evaluations per workload
bottom_5 = (
    df_sorted.groupby("workload", sort=False)
    .tail(5)[cols]
    .sort_values(["workload", "overall_score"])
    .reset_index(drop=True)
)

# Show full text in columns without truncation
pd.set_option("display.max_colwidth", None)

# Display all columns
pd.set_option("display.max_columns", None)

print("TOP 5 PER WORKLOAD")
display(top_5)

print("BOTTOM 5 PER WORKLOAD")
display(bottom_5)
```

```
TOP 5 PER WORKLOAD
```

	workload	model_name	overall_score	\
0	customer_response_drafting	llama_scout_17b	5.0	
1	customer_response_drafting	qwen3_32b	5.0	
2	customer_response_drafting	nvidia_nemotron_nano_12b	5.0	
3	customer_response_drafting	nova_lite	5.0	
4	customer_response_drafting	llama_scout_17b	5.0	
5	product_catalogue_extraction	llama_scout_17b	5.0	
6	product_review_summarization	llama_scout_17b	5.0	
7	product_review_summarization	qwen3_32b	5.0	
8	product_review_summarization	nova_lite	5.0	
9	product_review_summarization	qwen3_32b	5.0	
10	product_review_summarization	llama_scout_17b	5.0	
11	customer_issue_classification	qwen3_32b	5.0	
12	customer_issue_classification	nova_lite	5.0	

```
13  customer_issue_classification                nova_lite           5.0
14  customer_issue_classification                qwen3_32b           5.0
15  customer_issue_classification  nvidia_nemotron_nano_12b           5.0
16   product_catalogue_extraction          llama_scout_17b           4.6
17   product_catalogue_extraction          llama_scout_17b           4.6
18   product_catalogue_extraction  nvidia_nemotron_nano_12b           4.6
19   product_catalogue_extraction          llama_scout_17b           4.6
```

```
                                                                      ␣
  ↪                                                                   ␣
  ↪                                                                   ␣
  ↪         overall_comment
0                                                                     ␣
  ↪               The model-generated output effectively addresses the␣
  ↪customer's concern with empathy, professionalism, and helpfulness, while␣
  ↪adhering to policy and factual accuracy.
1                                                                     ␣
  ↪   The model-generated output effectively addresses the customer's concern␣
  ↪with empathy, professionalism, and helpfulness, while also adhering to company␣
  ↪policy and factual accuracy.
2                                                                     ␣
  ↪           The model-generated output effectively addresses the customer's␣
  ↪concern with empathy, professionalism, and clarity, while adhering to company␣
  ↪policy and factual accuracy.
3                                                                     ␣
  ↪        The model-generated output effectively addresses the customer's␣
  ↪concern with empathy, professionalism, and helpfulness, while adhering to␣
  ↪company policy and factual accuracy.
4                                                The model-generated output␣
  ↪effectively addresses the customer's concerns, demonstrates empathy, uses␣
  ↪professional language, provides helpful next steps, complies with company␣
  ↪policies, and is factually accurate.
5                                                                     ␣
  ↪                  The model-generated output is excellent, meeting all␣
  ↪evaluation criteria with high accuracy, completeness, relevance, hallucination␣
  ↪control, and format compliance.
6                                                                     ␣
  ↪                        The generated output is highly accurate, complete,␣
  ↪and concise, capturing the essence of the original review without introducing␣
  ↪any unsupported information.
7                                                                     ␣
  ↪                                      The model-generated output is␣
  ↪excellent, accurately summarizing the key points of the review with no␣
  ↪inaccuracies or unsupported information.
```

```
8                                                
```

```
19  The model-generated output is highly accurate, relevant, and well-formatted.␣
 ↪It adheres to the required JSON structure and avoids inventing missing data.␣
 ↪The only minor drawback is the absence of the 'price' field, which is not␣
 ↪provided in the original input.
```

BOTTOM 5 PER WORKLOAD

	workload	model_name	overall_score	\
0	customer_issue_classification	nova_lite	2.00	
1	customer_issue_classification	qwen3_32b	2.00	
2	customer_issue_classification	qwen3_32b	2.40	
3	customer_issue_classification	llama_scout_17b	2.60	
4	customer_issue_classification	llama_scout_17b	4.00	
5	customer_response_drafting	nvidia_nemotron_nano_12b	4.67	
6	customer_response_drafting	nvidia_nemotron_nano_12b	4.67	
7	customer_response_drafting	llama_scout_17b	4.67	
8	customer_response_drafting	nova_lite	4.67	
9	customer_response_drafting	llama_scout_17b	4.83	
10	product_catalogue_extraction	nova_lite	3.60	
11	product_catalogue_extraction	nova_lite	3.60	
12	product_catalogue_extraction	nova_lite	3.60	
13	product_catalogue_extraction	nova_lite	3.60	
14	product_catalogue_extraction	nova_lite	3.60	
15	product_review_summarization	qwen3_32b	4.67	
16	product_review_summarization	qwen3_32b	4.67	
17	product_review_summarization	nvidia_nemotron_nano_12b	4.67	
18	product_review_summarization	qwen3_32b	4.67	
19	product_review_summarization	nvidia_nemotron_nano_12b	4.67	

```
                                                                              ␣
 ↪                                                                            ␣
 ↪                                                                            ␣
 ↪               overall_comment
0                                                                             ␣
 ↪                                                  The model failed to␣
 ↪correctly classify the issue and provided an incorrect explanation, but it did␣
 ↪follow the required output format.
1                                                                             ␣
 ↪                                                    The model failed to␣
 ↪correctly classify the issue and provided an incorrect explanation, but it did␣
 ↪follow the required JSON format.
2                                                                             ␣
 ↪                          The model's output has some issues with␣
 ↪classification accuracy and information handling, but it correctly identifies␣
 ↪the urgency and follows the required format.
```

```
3                                                                              ␣
 ↪                      The model's output is partially correct but has␣
 ↪significant issues with classification accuracy and reasoning quality due to␣
 ↪an incorrect assumption about the product.
4                                                                              ␣
 ↪                       The model's response is well-structured and accurately␣
 ↪classifies the issue, though it could improve in handling discrepancies␣
 ↪between product details and the issue.
5                                                 The model-generated output is␣
 ↪highly effective in addressing the customer's concerns with professionalism␣
 ↪and helpfulness. It slightly falls short in factuality due to the lack of␣
 ↪exact dimensions for phone compatibility.
6                    The model-generated output is highly effective in addressing␣
 ↪the customer's questions with clear, professional, and factual information.␣
 ↪The only minor deduction is for the slight lack of empathy, as the response␣
 ↪could be slightly more personalized.
7   The model-generated output is highly effective in addressing the customer's␣
 ↪query with relevance, empathy, professionalism, and factual accuracy. It␣
 ↪slightly lacks in providing additional actionable next steps, which could␣
 ↪further enhance the customer's experience.
8       The model-generated output is highly effective in addressing the␣
 ↪customer's concern with empathy, professionalism, and factual accuracy. The␣
 ↪only minor area for improvement is providing more specific next steps, such as␣
 ↪contact information for the support team.
9                                                                      The␣
 ↪response is well-crafted, addressing the customer's questions with clarity and␣
 ↪professionalism. It could be slightly improved by providing more specific␣
 ↪recommendations for reaching the printer.
10                                                                             ␣
 ↪                                      The model performed well in accuracy,␣
 ↪relevance, and format compliance but fell short in completeness by not␣
 ↪extracting all available product fields.
11                                                                             ␣
 ↪                                The model performed well in terms of accuracy,␣
 ↪relevance, and format compliance but fell short in completeness by not␣
 ↪extracting all available product fields.
12                                                                             ␣
 ↪                                                  The model performed well in␣
 ↪accuracy, relevance, and format compliance but fell short in completeness by␣
 ↪missing several relevant fields.
13                                                                             ␣
 ↪                                      The model performed well in terms of␣
 ↪accuracy, relevance, and format compliance but fell short in completeness by␣
 ↪missing several key product fields.
```

```
14                                                                                ␣
 ↪                                                  The model performed well in␣
 ↪accuracy, relevance, and format compliance but missed several relevant product␣
 ↪fields, affecting completeness.
15                                                                          The␣
 ↪model-generated output is highly accurate, complete, and free from␣
 ↪hallucinations. It effectively captures the essence of the original review and␣
 ↪presents it in a clear and concise manner.
16                                                                 The␣
 ↪model-generated output is highly accurate, complete, and well-aligned with the␣
 ↪original review. It effectively captures the sentiment and themes while␣
 ↪maintaining conciseness and avoiding hallucinations.
17                                                                          The␣
 ↪model-generated output is highly accurate, complete, and free from␣
 ↪hallucinations. It effectively captures the essence of the original review and␣
 ↪presents it in a clear and concise manner.
18                                      The model-generated output is highly␣
 ↪accurate, complete, and free from hallucinations. It effectively captures the␣
 ↪sentiment and themes of the original review. The only minor area for␣
 ↪improvement is the conciseness of the summary.
19                                                          The model-generated␣
 ↪output is highly accurate, complete, and well-aligned with the original review.
 ↪ It effectively captures the negative sentiment and key themes without␣
 ↪introducing any unsupported information.
```

Based on the distributions observed across the tasks we can see we have actual spread across the product catalogue extraction and customer issue classification. product category extraction had more of a normal spread while product catalogue extraction was more impacted by outliers. This is perhaps to the rather simple, almost binarized nature of the product catalogue task.

[60]:
```
%matplotlib inline
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Clean labels for readable chart titles and legends
df_plot = df_full.copy()
df_plot["workload_clean"] = df_plot["workload"].str.replace("_", " ").str.
 ↪title()
df_plot["model_clean"] = df_plot["model_name"].str.replace("_", " ")

sns.set_theme(style="whitegrid", context="notebook")
palette = sns.color_palette("Set2", n_colors=df_plot["model_clean"].nunique())

# Define the 4 raw metrics to visualize
metrics = [
    ("overall_score", "Quality Score Distribution (1-5)", "Overall Score"),
```

```
    ("latency_seconds", "End-to-End Latency Distribution (Seconds)", "Latency␣
 ↪(Seconds)"),
    ("input_tokens", "Input Token Count Distribution", "Input Tokens"),
    ("output_tokens", "Output Token Count Distribution", "Output Tokens"),
]

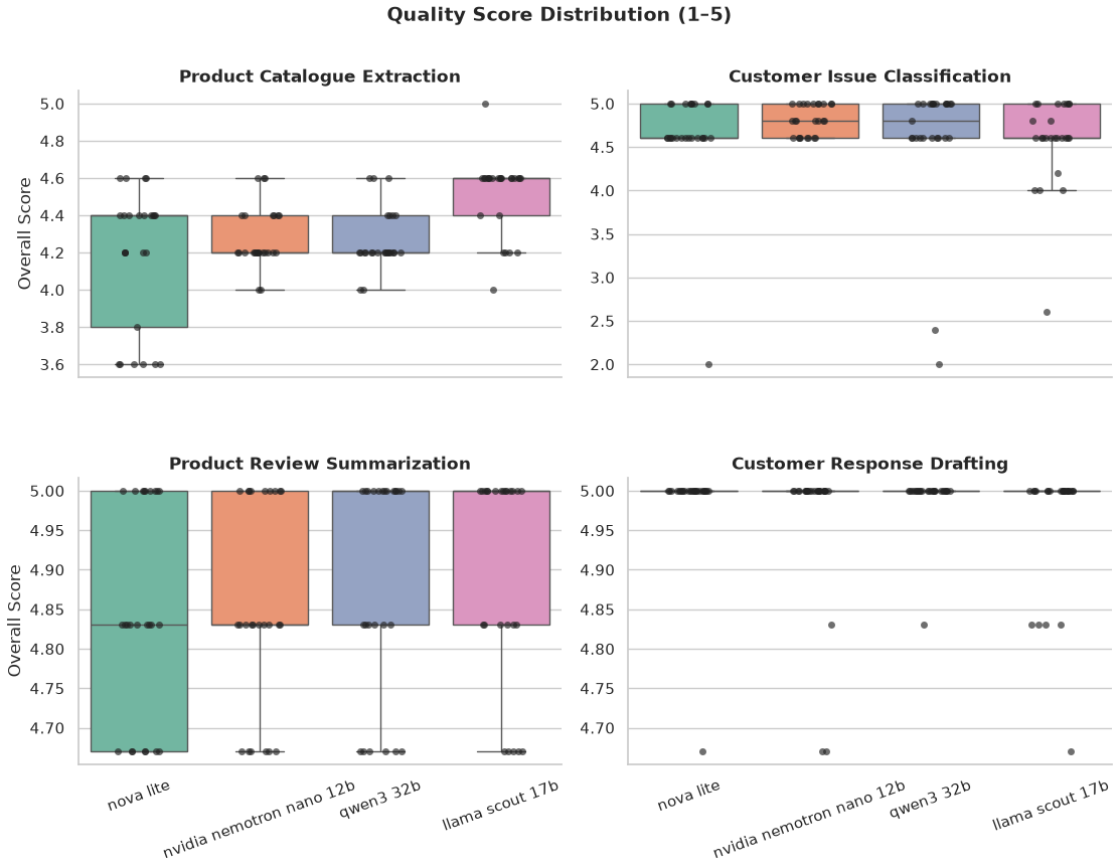
# Generate one multi-panel figure per metric (split by workload)
for col, title, ylabel in metrics:
    g = sns.catplot(
        data=df_plot,
        x="model_clean",
        y=col,
        hue="model_clean",
        col="workload_clean",
        col_wrap=2,
        kind="box",
        palette=palette,
        height=4.2,
        aspect=1.35,
        sharey=False,       # Allows each workload to scale to its own range
        showfliers=False,   # Hides duplicate outlier diamonds since stripplot␣
 ↪shows all points
        legend=False,
    )

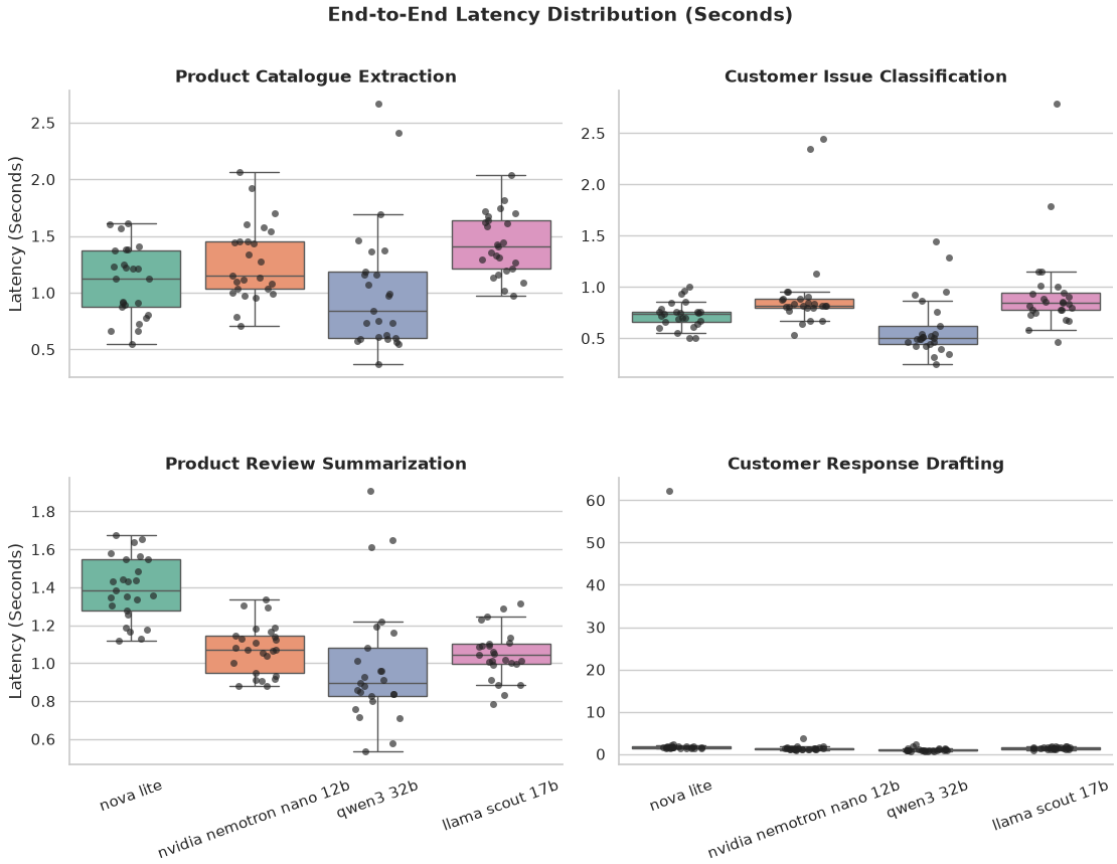
    # Overlay individual evaluation runs (25 dots per model)
    g.map_dataframe(
        sns.stripplot,
        x="model_clean",
        y=col,
        color="#222222",
        size=5,
        jitter=0.18,
        alpha=0.65,
    )

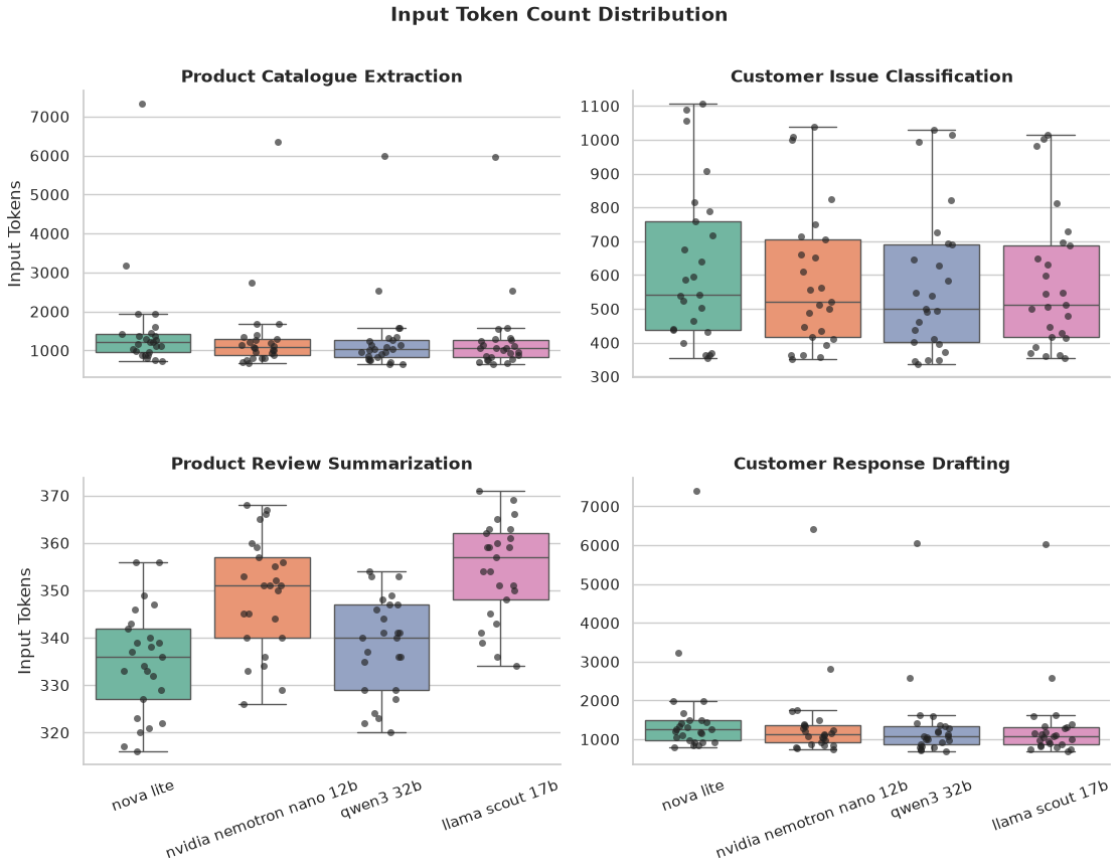
    g.set_titles("{col_name}", fontweight="bold", fontsize=11)
    g.set_axis_labels("", ylabel)

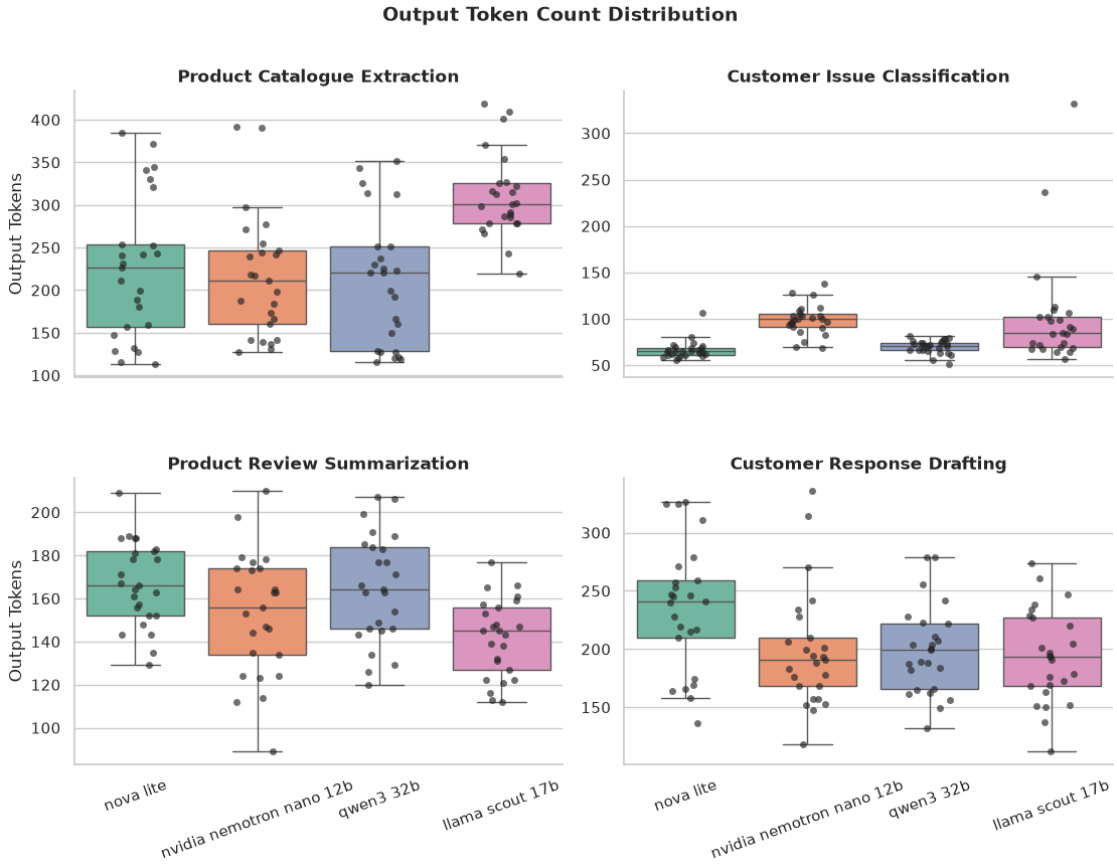
    # Rotate model names on x-axis so they never overlap
    for ax in g.axes.flat:
        ax.tick_params(axis="x", rotation=20)

    plt.subplots_adjust(top=0.88, hspace=0.35)
    g.fig.suptitle(title, fontsize=14, fontweight="bold")
    plt.show()
```


Quality Score Distribution (1–5)
Product Catalogue Extraction
Customer Issue Classification
Product Review Summarization
Customer Response Drafting
Overall Score
nova lite
nvidia nemotron nano 12b
qwen3 32b
llama scout 17b


End-to-End Latency Distribution (Seconds)
Product Catalogue Extraction
Customer Issue Classification
Product Review Summarization
Customer Response Drafting
Latency (Seconds)
nova lite
nvidia nemotron nano 12b
qwen3 32b
llama scout 17b


Input Token Count Distribution
Product Catalogue Extraction
Customer Issue Classification
Product Review Summarization
Customer Response Drafting
Input Tokens
nova lite
nvidia nemotron nano 12b
qwen3 32b
llama scout 17b


Output Token Count Distribution
Product Catalogue Extraction
Customer Issue Classification
Product Review Summarization
Customer Response Drafting
Output Tokens
nova lite
nvidia nemotron nano 12b
qwen3 32b
llama scout 17b